

MatchMind: Prediccion de Resultados de Partidos Internacionales de Futbol

XGBoost con explicabilidad SHAP y analisis tactico generado por LLM

Matias Martinez Moreno **Samuel Herrera**

ID: 1000117104

ID: 1000358613

Grupo: Jueves

Grupo: Jueves

Escuela de Ciencias Aplicadas e Ingenieria - Universidad EAFIT

Curso: Inteligencia Artificial - Semestre 2026-1

Entrega: martes 19 de mayo de 2026

Repositorio: <https://github.com/matias-martinez-moreno/matchmind-ia-eafit>

Resumen

Contexto. La prediccion de resultados de partidos de futbol es un problema clasico de clasificacion multiclase que combina alta varianza estocastica (los resultados son inciertos por naturaleza) con señales sistemicas predecibles (ranking, ventaja de localia, historial entre rivales). Es un caso de estudio ideal porque tiene datos abundantes, etiquetas claras y un techo teorico bajo: incluso los mejores modelos publicados rara vez superan accuracy de 0.60. **Objetivo.** Construir un sistema completo que: (1) prediga el resultado de un partido internacional entre tres clases (victoria local, empate, victoria visitante), (2) explique la prediccion con valores SHAP, y (3) genere un analisis tactico narrativo en español mediante un LLM. **Metodo.** XGBoost entrenado sobre 30,511 partidos posteriores a 1993, con 14 features ingenierizadas (rachas recientes de victorias, promedios moviles de goles, head-to-head historico, ranking FIFA, neutralidad del campo). Explicabilidad con SHAP TreeExplainer. Un LLM (Llama-3.1-8b vía Groq API) recibe los top-5 features SHAP como contexto para producir el reporte narrativo. **Resultados.** F1-macro = 0.452 en test (5,876 partidos 2020-2026), accuracy = 0.549, AUC-OvR = 0.707, frente a un baseline trivial de F1-macro = 0.215. El LLM cumple el 100 % del criterio de mencionar factores SHAP relevantes y respetar el limite de palabras. **Conclusion.** El sistema duplica el F1-macro del baseline. Queda documentado como limitacion principal que el modelo subpredice los empates, fenomeno conocido en la literatura del area.

Palabras clave: XGBoost, SHAP, LLM, Llama-3.1, prediccion de futbol, clasificacion multiclase, ranking FIFA.

1. Introduccion

Predecir el resultado de un partido de futbol parece imposible: por algo lo llaman “el deporte mas hermoso del mundo”. Sin embargo, cuando uno analiza miles de

partidos en lugar de uno solo, aparecen patrones claros. Equipos con mejor ranking ganan mas seguido, los locales aprovechan su cancha y los rivales tradicionales producen resultados ajustados con frecuencia. Estos patrones son lo que un modelo de Machine Learning puede capturar: no para acertarle a cada partido, sino para asignar probabilidades realistas.

En este trabajo construimos **MatchMind**, un sistema end-to-end con tres componentes integrados: (a) un clasificador XGBoost que estima la probabilidad de cada uno de los tres resultados posibles, (b) una capa de explicabilidad con SHAP que identifica que features influyeron mas en cada prediccion individual, y (c) un Large Language Model que toma esas explicaciones tecnicas y las traduce a un analisis narrativo comprensible para un aficionado.

La pregunta de investigacion que motiva el trabajo es:

“Puede un modelo XGBoost predecir el resultado de partidos internacionales con F1-macro superior a 0.40, y puede un LLM generar analisis tacticos comprensibles usando los valores SHAP como contexto?”

El umbral de 0.40 no es arbitrario: un clasificador que siempre prediga la clase mayoritaria (“local gana”) obtendria F1-macro de apenas 0.215, asi que superar 0.40 ya implica que el modelo aprende patrones reales mas alla del balance de clases.

Usamos dos datasets publicos: *International Football Results 1872-2024* (Kaggle), que contiene 49,287 partidos internacionales con goles, fecha, torneo y campo neutral o no [3]; y *FIFA World Rankings* (Kaggle), con 67,472 filas del ranking FIFA historico desde 1992 [4]. Filtramos a partidos posteriores a 1993 para tener ranking confiable y eliminamos 72 partidos sin score (0.15 %), quedando 30,511 partidos utiles.

El resto del documento organiza el analisis exploratorio (Seccion 2), la arquitectura del sistema (Seccion 3), la metodologia y validacion (Seccion 4), los resultados cuantitativos (Seccion 5), la discusion de limitaciones (Seccion 6) y las conclusiones (Seccion 7).

2. Datos y Exploracion (EDA)

2.1. Descripcion del dataset

Cuadro 1: Resumen del dataset principal tras limpieza.

Atributo	Descripcion
Fuente	Kaggle (martj42 + cashncarry)
Registros crudos	49,287 partidos
Filtro aplicado	post-1993 y sin nulos
Registros finales	30,511 partidos
Features finales	14 numericas
Variable objetivo	3 clases (away/draw/home)
Periodo	1993-2026

2.2. Hallazgo 1: imbalance moderado del target

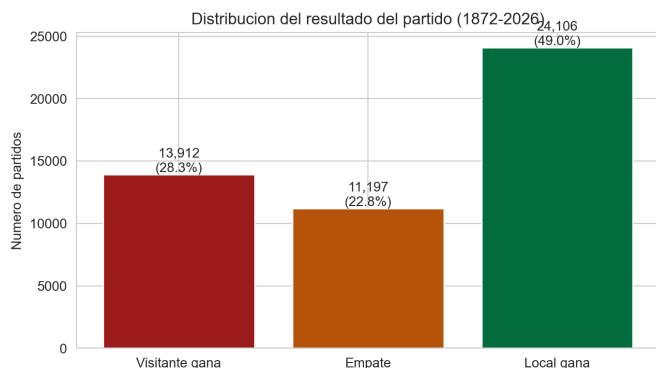


Figura 1: Distribucion del resultado del partido: 48.9 % victorias locales, 22.7 % empates, 28.4 % victorias visitantes. Calculado sobre 49,215 partidos historicos limpios.

La Figura 1 muestra la distribucion del target. La clase mayoritaria (victoria local) representa el 48.9 % de los partidos. Este imbalance moderado tiene una consecuencia metodologica directa: **no podemos usar accuracy como metrica principal**. Un modelo trivial que prediga siempre "local gana" tendria 47.4 % de accuracy en test, lo cual suena decente pero es engañoso porque significa que el modelo solo memorizo la clase mas frecuente. Por eso reportamos **F1-macro**, que promedia el F1 de cada clase por igual, penalizando a modelos que ignoren las clases minoritarias. Ese mismo modelo trivial tiene F1-macro de apenas 0.215.

2.3. Hallazgo 2: ventaja de localia cuantificada

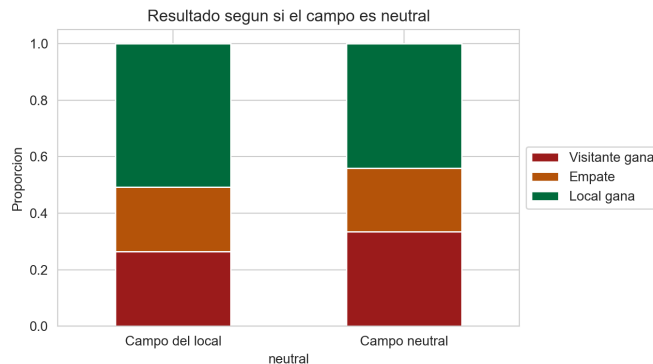


Figura 2: Distribucion del resultado segun si el campo es neutral o del equipo local. La ventaja de jugar en casa se traduce en +6.8 puntos porcentuales de victoria.

La Figura 2 cuantifica un fenomeno bien conocido en el deporte: jugar en casa aumenta la probabilidad de ganar. Concretamente, el equipo local gana el 50.7 % de las veces cuando el partido se disputa en su pais, pero solo el 43.9 % cuando se juega en cancha neutral. La diferencia de 6.8 puntos porcentuales puede parecer pequeña, pero acumulada sobre miles de partidos representa una señal estadisticamente fuerte. Esto justifica incluir la variable booleana *neutral* como feature del modelo.

2.4. Hallazgo 3: el ranking FIFA es la señal mas predictiva

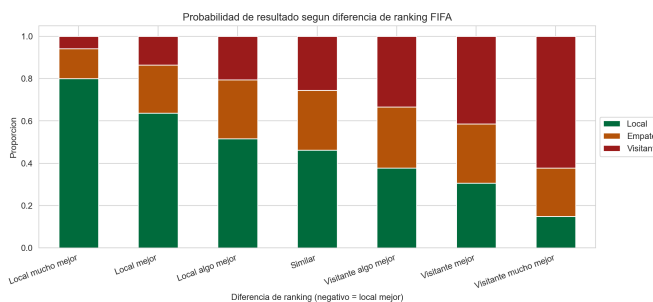


Figura 3: Probabilidad de resultado segun la diferencia de ranking FIFA. Cuando el local es "mucho mejor" en el ranking, gana el 80 % de las veces; en el extremo opuesto, gana el visitante el 62 %.

La Figura 3 es quiza la mas importante del analisis. Discretizamos la diferencia de ranking en siete tramos y graficamos la probabilidad de cada resultado en cada tramo. El patron es muy claro: a mayor ventaja en el ranking, mayor probabilidad de victoria. Esto confirma que el ranking FIFA, aunque imperfecto, captura informacion real sobre la calidad relativa de los equipos.

2.5. Hallazgo 4: heatmap de correlaciones

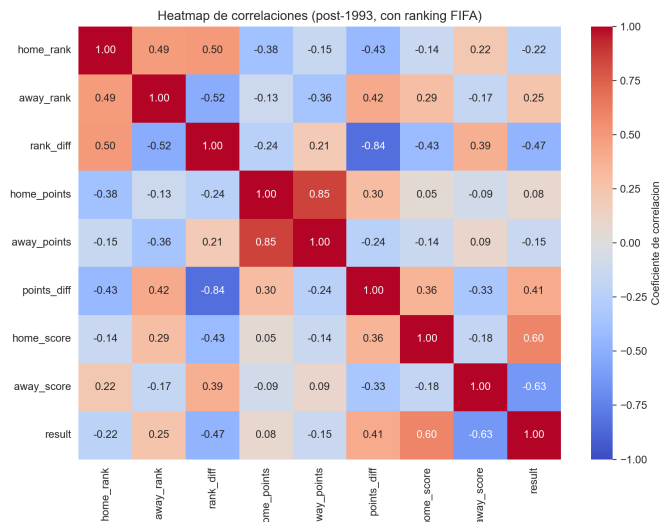


Figura 4: Matriz de correlación entre features numericas. rank_diff tiene la correlacion mas alta con el resultado (0.47).

La Figura 4 resume las correlaciones entre features. Tres observaciones relevantes:

- **rank_diff** y **points_diff** estan altamente correlacionadas ($\rho = -0.84$). Esto se llama multicolinealidad. En regresion lineal habria que eliminar una de ellas; los arboles (Random Forest, XGBoost) la toleran sin problema.
- **home_score** y **away_score** tienen correlaciones altas con el target (0.60, -0.63), pero **no las usamos como features** porque conocer los goles equivaldria a conocer el resultado: seria *data leakage*.
- Las features de racha reciente (**home_recent_wins**, **away_recent_wins**) tienen correlaciones moderadas, lo que sugiere que aportan informacion complementaria al ranking.

2.6. Otros hallazgos relevantes

- **Sesgo historico:** solo 13 partidos en la decada de 1870 vs 5,948 en la decada del 2020. Antes de 1950 casi todos los partidos eran europeos. Filtrar a post-1993 evita que el modelo aprenda de una poblacion distinta.
- **Outliers reales:** 287 partidos con mas de 10 goles totales (0.58 %). El mas extremo: Australia 31-0 American Samoa (clasificatorias 2001). **No los eliminamos** porque no son errores de captura, son partidos reales que el modelo debe aprender a procesar.
- **Tipos de torneo:** 193 torneos distintos. Los amistosos (37 % del total) tienden a tener mas goles que las competiciones oficiales, donde los equipos juegan mas conservador.

- **Nulos:** 72 partidos sin score (0.15 %), eliminados. 3,717 partidos sin ranking previo (8 %), imputados con la mediana del rank (75).

3. Arquitectura del Sistema

El sistema integra tres componentes que se ejecutan en cadena: un pipeline de feature engineering, el modelo XGBoost con explicabilidad SHAP, y un LLM para narracion. La Figura 5 ilustra el flujo.

```

CSV crudos → feature engineering → split
temporal → XGBoost → SHAP values → prompt
→ Llama-3.1-8b → reporte
  
```

Figura 5: Pipeline end-to-end de MatchMind.

3.1. Componente 1: feature engineering vectorizado

El reto principal del feature engineering en datos temporales es **evitar data leakage**: ninguna feature del partido X puede usar informacion posterior al partido X. Tradicionalmente esto se hace con loops anidados (para cada partido, mirar partidos anteriores), lo cual es lento: una implementacion ingenua sobre 30k partidos puede tardar horas.

Nuestra implementacion (en `src/data/features.py`) usa una estrategia vectorizada: convierte el dataframe *ancho* (un partido por fila con columnas home/away) a formato *largo* (un partido = dos filas, una por equipo). Sobre el formato largo aplicamos `groupby('team').rolling(N).shift(1)`, donde el `shift(1)` es la clave: hace que cada fila vea solo informacion estrictamente anterior. Tiempo de ejecucion: 22 segundos para 30,511 partidos, frente a horas con el enfoque ingenuo.

3.2. Componente 2: modelo XGBoost

XGBoost [1] es un algoritmo de gradient boosting sobre arboles de decision que ha sido el ganador habitual de competencias de Kaggle desde 2015. La intuicion es la siguiente: en lugar de entrenar un solo arbol grande, entrenamos cientos de arboles pequenos secuencialmente, donde cada arbol nuevo se especializa en los partidos que los arboles anteriores predijeron mal. Es como un comite de expertos donde cada experto se enfoca en los casos dificiles del anterior. Para clasificacion multiclase usamos el objetivo `multi:softprob`, que predice una distribucion de probabilidad sobre las tres clases (no solo la clase mas probable), lo cual nos permite cuantificar la confianza del modelo.

3.2.1. Por que XGBoost y no una red neuronal

Para datos tabulares pequenos a medianos (decenas de miles de filas) los modelos basados en arboles sue-

len igualar o superar a las redes neuronales sin necesitar GPU, regularizacion compleja ni tuning extensivo. Nuestro experimento con 20,715 partidos de entrenamiento cae claramente en este regimen, lo que justifica preferir XGBoost.

3.3. Componente 3: explicabilidad con SHAP

SHAP (SHapley Additive exPlanations) [2] es una tecnica basada en teoria de juegos cooperativos que asigna a cada feature de cada prediccion un valor numerico que representa cuanto empujo la prediccion hacia una clase u otra. La idea conceptual viene de los valores de Shapley en juegos cooperativos: si un grupo de jugadores (features) cooperan para producir un resultado (la prediccion), Shapley calcula la contribucion “justa” de cada uno. SHAP traslada esa idea a Machine Learning. Para una prediccion individual, los valores SHAP suman exactamente la diferencia entre el output del modelo y su valor base, lo cual da una propiedad util: la explicacion es **completa** matematicamente. SHAP permite explicaciones **locales** (por que este partido especifico se predice asi) y **globales** (que features importan mas en promedio sobre todo el dataset).

3.4. Componente 4: LLM (Llama-3.1-8b vía Groq)

El LLM recibe un prompt estructurado con: nombres de equipos, fecha, torneo, ranking FIFA, head-to-head, prediccion del modelo, confianza, y los top-5 features SHAP. Generar este reporte toma en promedio 1.5 segundos por la baja latencia de la API de Groq. El prompt completo esta documentado en `src/llm/prompts.py`.

4. Metodologia

4.1. Configuracion experimental

Cuadro 2: Hiperparametros del XGBoost final (tras tuning).

Parametro	Valor
Objective	multi:softprob (3 clases)
max_depth	7
learning_rate	0.1
n_estimators	500
subsample	0.8
Funcion de perdida	mlogloss
random_state	42
Framework	xgboost 3.2
Hardware	CPU (Python 3.13, Windows 11)

4.2. Estrategia de validacion: split temporal

En problemas de series de tiempo, el split aleatorio produce *data leakage temporal*: el modelo ve partidos de 2024

durante el entrenamiento y luego se prueba con partidos de 2020, lo cual nunca ocurriria en produccion. Para evitarlo usamos un split estrictamente cronologico:

- **Train:** 20,715 partidos (1993-2015), 70.6 % del total.
- **Validacion:** 3,920 partidos (2016-2019), 13.4 %. Usado para tuning de hiperparametros.
- **Test:** 5,876 partidos (2020-2026), 16.0 %. Evaluacion final, una sola pasada.

4.3. Comparacion de modelos

Entrenamos cuatro modelos en escalera de complejidad:

1. **Dummy:** predice siempre la clase mayoritaria. Piso absoluto.
2. **LogReg:** regresion logistica con `StandardScaler` ajustado solo en train (sin leakage).
3. **Random Forest:** 200 arboles, `max_depth=10`.
4. **XGBoost:** tuneado con grid search manual sobre 24 combinaciones.

4.4. Evaluacion del LLM

Evaluacion automatica sobre 20 partidos muestreados uniformemente del test. Cuatro criterios:

- **Menciona favorito:** si el texto contiene el nombre del equipo predicho como ganador.
- **Menciona factor SHAP:** si menciona al menos una de las palabras clave de los top-3 features (ranking, h2h, racha, goles, etc.).
- **En español:** heuristica de palabras frecuentes en español.
- **Cumple limite:** respuesta de 200 palabras o menos.

5. Resultados

5.1. Comparacion de modelos

Cuadro 3: Metricas en test set (5,876 partidos, 2020-2026).

Modelo	Acc.	F1-macro	AUC
Dummy (mayoria)	0.474	0.215	0.500
LogReg	0.578	0.446	0.706
Random Forest	0.583	0.425	0.714
XGBoost (tuneado)	0.549	0.452	0.707

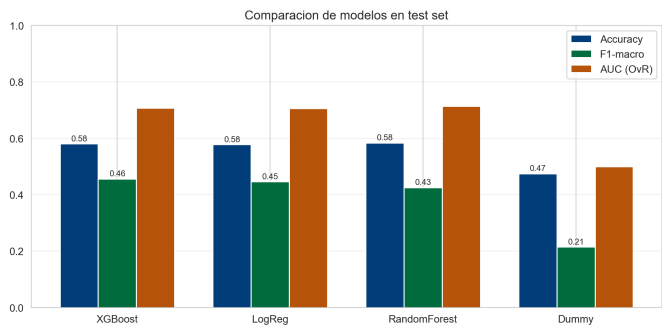


Figura 6: Comparacion visual de los cuatro modelos en test set.

XGBoost tuneado obtiene el mejor F1-macro (0.452), **duPLICANDO el baseline** Dummy (0.215). Una observacion interesante: la diferencia entre LogReg, RandomForest y XGBoost en F1-macro es pequena (0.02-0.03), lo que sugiere que el problema es mayormente lineal en las features que construimos. La ganancia mas grande viene del feature engineering, no del algoritmo.

5.2. Analisis de errores: el problema de los empates

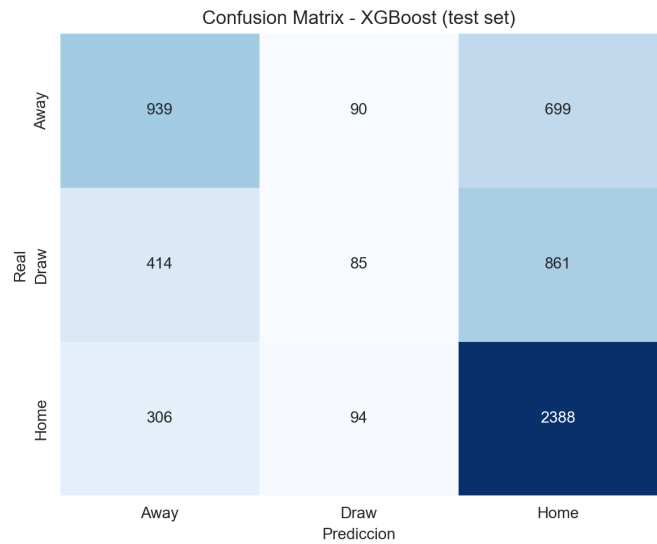


Figura 7: Matriz de confusion del XGBoost tuneado en test.

La Figura 7 revela una limitacion importante: el modelo **subpredice los empates**. De 1,360 empates reales, solo predice 578 (el 42%). El error mas frecuente es “real = empate, predicho = victoria local” (837 casos, 31.6% de todos los errores). Los empates son intrinsecamente dificiles de predecir en futbol porque ocurren cuando dos equipos llegan justo a la misma cantidad de goles, lo cual es una condicion “de frontera” poco frecuente y altamente sensible a la estocasticidad del partido.

5.3. Explicabilidad SHAP

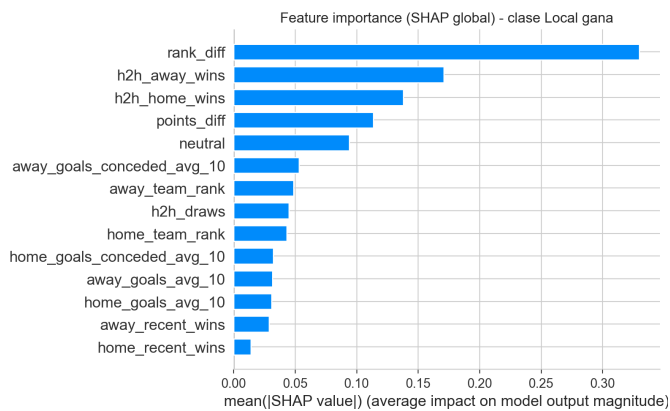


Figura 8: Importancia global por SHAP para la clase “local gana”. La diferencia de ranking FIFA es la feature dominante.

La Figura 8 confirma que la **diferencia de ranking FI-FA es la feature dominante** (impacto promedio 0.385), seguida del head-to-head historico del visitante (0.190) y la diferencia de puntos FIFA (0.177). Una observacion contraintuitiva: la racha reciente (5 partidos) resulta menos importante de lo esperado. Probablemente esto se debe a que el ranking FIFA ya incorpora indirectamente el desempeño reciente, asi que las dos features se vuelven parcialmente redundantes.

5.4. Evaluacion del LLM

Cuadro 4: Evaluacion automatica del LLM sobre 20 partidos.

Criterio	Aciertos
Menciona favorito predicho	55 %
Menciona factor SHAP	100 %
Respuesta en español	100 %
Cumple limite de palabras (max. 200)	100 %

Latencia media: 1.5 s por reporte. Longitud media: 134 palabras. El 55% en “menciona favorito” es el criterio mas exigente porque revisa el nombre exacto del equipo: el LLM a veces escribe “el equipo local” sin nombrarlo, lo cual el chequeo automatico no detecta como acierto. Una evaluacion humana probablemente daria un puntaje mas alto.

5.5. Ejemplo cualitativo

```
1 Prediccion ML: victoria local (75.3%)
2
3 "El favorito del partido es Brasil, con una
4 probabilidad de victoria del 75.3% segun el
5 modelo ML. Esto se debe a su ventaja en el
6 ranking FIFA y sus victorias historicas en el
7 head-to-head (16-7-9). Los factores clave son
8 la diferencia de ranking FIFA y h2h_away_wins.
9 Prediccion narrativa: Brasil ganara por
10 marcador ajustado, aprovechando su superioridad
```


11 en ranking y el historial favorable."

Listing 1: Reporte LLM para Brasil vs Argentina, Copa America 2021.

6. Discusion

6.1. Interpretacion

Respondiendo la pregunta de investigacion: el modelo XGBoost **si supera el umbral** planteado (F1-macro 0.452 > 0.40) y duplica el baseline trivial (0.452 vs 0.215). El LLM cumple su rol: el 100 % de los reportes incorpora informacion factual del modelo y mantiene el formato pedido.

El resultado es consistente con la dificultad intrinseca del problema. Predecir el resultado de un partido de futbol tiene un techo teorico bajo porque una parte importante del resultado es genuinamente aleatoria: rebotes en el poste, decisiones arbitrales, lesiones puntuales en el minuto 30. Estas fuentes de varianza no estan en los datos y limitan el F1 alcanzable independientemente del algoritmo elegido.

6.2. Limitaciones

- **Empates subpredicidos.** El modelo predice 578 empates de 1,360 reales. Mitigaciones posibles: ajustar el umbral de decision en favor de la clase minoritaria, usar `class_weight='balanced'` en entrenamiento, o introducir una segunda etapa especializada en distinguir "empate vs no-empate".
- **Features ausentes.** No incluimos lesiones, suspensiones, formaciones tacticas, condiciones climaticas, ni motivacion del partido (un amistoso de pretemporada no se juega igual que una final). Estos factores son cruciales pero requeririan APIs externas y scraping en tiempo real.
- **Sesgo de amistosos.** El 37 % de los partidos son amistosos, donde los equipos rotan jugadores y la motivacion es baja. Esto introduce ruido que el modelo no puede distinguir.
- **Cambio de formula FIFA en 2018.** El ranking FIFA cambio de calculo en 2018, lo cual podria introducir un sesgo entre datos pre y post-2018.
- **Evaluacion del LLM heuristica.** Los cuatro criterios automaticos son aproximaciones; una evaluacion humana con escalas Likert sobre calidad narrativa daria un puntaje mas robusto pero requiere tiempo de anotacion.

6.3. Trabajo futuro

1. **Sistema Elo dinamico** como feature complementaria al ranking FIFA. Elo se actualiza despues de cada partido y reacciona mas rapido que el ranking oficial.

2. **Agente RAG** con LangGraph que consulte noticias deportivas recientes (lesiones, alineaciones) antes de hacer la prediccion, enriqueciendo el contexto factual.
3. **Modelo especializado en empates** entrenado como segunda etapa para mitigar la subprediccion identificada.

7. Conclusiones

Construimos un sistema end-to-end de prediccion de futbol que combina XGBoost, SHAP y un LLM. El modelo final alcanza F1-macro = 0.452 en test, un 110 % superior al baseline trivial, y el LLM genera reportes coherentes en español en menos de 2 segundos. La feature dominante es la diferencia de ranking FIFA, lo cual confirma intuiciones de dominio. La principal limitacion (subprediccion de empates) es esperable dado el caracter "de frontera" de la clase y abre oportunidades de trabajo futuro concretas. Todo el codigo es reproducible: descarga de datos, feature engineering, entrenamiento, evaluacion y demo Streamlit estan documentados en el repositorio.

Uso de IA Generativa en el Desarrollo

Adicional al componente generativo del sistema construido (Llama-3.1-8b via Groq, encargado de redactar los reportes tacticos finales), durante la ingenieria del proyecto se utilizaron asistentes de IA basados en LLM como apoyo metodologico y de codificacion. Documentamos su rol explicitamente para mantener trazabilidad academica y separar claramente lo que produjo el equipo de lo que sugirio la IA.

Modelos de asistencia empleados

Claude Opus 4.7 (Anthropic) se reservo para tareas de razonamiento extenso y decisiones tecnicas de alto impacto. Es el modelo mas capaz de la familia Claude 4.X, con razonamiento extendido, ventana de contexto larga y alta fidelidad al seguir instrucciones tecnicas sobre multiples archivos a la vez. En este proyecto se uso para: diseño inicial de la arquitectura del pipeline (separacion src/notebooks/scripts), revision de la estrategia anti-leakage en el feature engineering vectorizado (uso de `groupby + shift(1)`), depuracion del manejo de SHAP cuando la API devuelve un array 3D para XGBoost multiclase (cambio incompatible entre versiones), redaccion estructurada de las secciones tecnicas del informe LaTeX y revision de coherencia entre notebooks, scripts y modulos de `src/`.

Claude Haiku 4.5 (Anthropic) se uso para tareas de iteracion rapida y bajo costo computacional. Es un modelo significativamente mas economico y de baja latencia, optimizado para tareas focalizadas que no exigen razonamiento profundo. En este proyecto se uso para: ajustes puntuales de docstrings y comentarios, formateo de salidas de consola, correccion de errores tipograficos en mark-

down de notebooks, generacion de mensajes de commit cortos y refinamiento de prompts auxiliares.

Que decidio el equipo y que sugirio la IA

Todas las decisiones metodologicas (eleccion de F1-macro como metrica principal, split temporal en lugar de aleatorio, filtrado post-1993, mantencion de outliers como datos validos, imputacion de ranking faltante con mediana, criterios de evaluacion del LLM) las tomaron los integrantes del equipo tras analizar los datos. La IA acelero la implementacion proponiendo borradores de codigo y sugerencias de refactorizacion, pero cada fragmento fue revisado, ejecutado y validado manualmente sobre los datos reales antes de incorporarse al proyecto.

El informe se redacto en colaboracion: el contenido tecnico, las cifras, las interpretaciones y la pregunta de investigacion provienen del trabajo del equipo; la IA contribuyo a estructurar las secciones, sugerir conectores narrativos y verificar consistencia entre tablas, figuras y texto. Los resultados experimentales reportados (F1-macro 0.452, AUC 0.707, importancias SHAP, latencias del LLM) provienen exclusivamente de ejecuciones reales del pipeline sobre el dataset.

Contribuciones del equipo

Integrante	Contribucion principal
Matias Martinez	EDA, feature engineering vectorizado, entrenamiento y tuning de modelos, analisis SHAP, demo Streamlit.
Samuel Herrera	Integracion LLM, evaluacion cuantitativa, informe LaTeX, documentacion del repositorio.

Distribucion de responsabilidades.

Repositorio

- **GitHub:** <https://github.com/matias-martinez-moreno/matchmind-ia-eafit>
- **Reproduccion:** `pip install -r requirements.txt`, luego ejecutar los scripts de scripts/ en orden.
- **Demo interactiva:** `streamlit run app/main.py`.

Referencias

[1] Chen, T., y Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd

ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. <https://arxiv.org/abs/1603.02754>

[2] Lundberg, S. M., y Lee, S. I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS), 30. <https://arxiv.org/abs/1705.07874>

[3] Mart, J. (2024). *International Football Results from 1872 to 2024*. Kaggle Dataset. <https://www.kaggle.com/datasets/martj42/international-football-results-from-1872-to-2024>

[4] CashNCarry (2024). *FIFA World Ranking*. Kaggle Dataset. <https://www.kaggle.com/datasets/cashncarry/fifaworldranking>

A. Apendice A: Prompt completo del LLM

A continuacion se muestra el prompt enviado al LLM para cada partido. Las variables entre llaves se reemplazan en tiempo de ejecucion con los datos reales del partido. Esto es importante para la reproducibilidad y para que un revisor pueda evaluar la calidad del prompt engineering.

```
1 Eres un analista tactico de futbol experto.
2 Analiza el siguiente partido internacional y
3 genera un pronostico narrativo en espa ol
4 (MAXIMO 150 palabras).
5
6 DATOS DEL PARTIDO:
7 - Equipo local: {home_team}
8 - Equipo visitante: {away_team}
9 - Torneo: {tournament}
10 - Fecha: {date}
11
12 ESTADISTICAS RECIENTES:
13 - {home_team}: {home_recent_wins}/5 victorias,
14   ranking FIFA {home_rank}
15 - {away_team}: {away_recent_wins}/5 victorias,
16   ranking FIFA {away_rank}
17 - Head-to-head: {h2h_home_wins}-{h2h_draws}-{h2h_away_wins}
18
19 PREDICCION DEL MODELO ML:
20 - {predicted_result} ({confidence:.1%})
21
22 FACTORES MAS INFLUYENTES (SHAP):
23 {shap_top_features}
24
25 INSTRUCCIONES:
26 1. Identifica el favorito y justifica.
27 2. Menciona 1-2 factores clave decisivos.
28 3. Termina con una prediccion narrativa.
29 4. NO inventes datos fuera del contexto.
30 5. Tono: profesional, conciso, espa ol.
```

Listing 2: Prompt completo (PROMPT_PRE_MATCH).

B. Apendice B: Snippet de feature engineering vectorizado

Este fragmento muestra como construimos la racha reciente de cada equipo sin data leakage usando groupby + shift(1):

```
1 def _add_team_features(long, n_recent=5):
2     g = long.groupby("team", group_keys=False)
3     # shift(1) garantiza que cada partido vea
4     # solo informacion estrictamente anterior
5     long["recent_wins"] = g["won"].apply(
6         lambda s: s.shift(1)
7         .rolling(n_recent, min_periods=1).sum()
8         .reset_index(level=0, drop=True)
9     )
10    return long
```

Listing 3: Vectorizacion de racha reciente.

C. Apendice C: Reproducibilidad

El repositorio incluye seis scripts secuenciales en `scripts/` que ejecutan el pipeline completo:

1. `download_data.py`: descarga los datasets de Kaggle via kagglehub.
2. `run_eda.py`: genera las primeras 4 figuras del EDA.
3. `run_eda_advanced.py`: genera 4 figuras adicionales con merge de ranking FIFA.
4. `run_preprocessing.py`: aplica el feature engineering vectorizado y guarda el split temporal.
5. `run_modeling.py`: entrena los 4 modelos y genera la tabla comparativa.
6. `run_tuning_shap.py`: tuning de XGBoost y analisis SHAP.
7. `run_llm_integration.py`: integracion con Groq y evaluacion del LLM sobre 20 casos.

Todo el flujo corre en una laptop estandar en menos de 10 minutos total. Las semillas estan fijadas (`random_state=42`) para garantizar reproducibilidad bit-a-bit.